



233

**UNIVERSITY OF COLOMBO, SRI LANKA****UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING****Bachelor of Science in Computer Science****First Year Examination - Semester 1 - 2023****SCS 1202 — Programming using C****TWO HOURS****To be completed by the candidate****Index Number**

--	--	--	--	--	--	--	--

Important Instructions

- This paper having 12 pages is in two parts, **Part A** and **Part B**. Part A consists of 20 MCQs and Part B consists of 5 questions. **The candidate must answer all questions.**
- **Write your answers only in the answer boxes provided** on this question paper.
- Do not tear off any part of this answer book. Under no circumstances may this book (or any part of this book), used or unused, be removed from the examination hall by a candidate.
- Questions appear on both sides of the paper. If a page is not printed, please inform the supervisor immediately.
- Any electronic device capable of storing and retrieving text including electronic dictionaries and mobile phones are **not allowed**.
- Calculators are **not allowed**.
- Please write your answers only in English.
- The C operator precedence table is given on page 12.

To be completed by the examiners

A	
1	
2	
3	
4	
5	
Total	

Part A

In each of the questions 1 to 20, pick one of the alternatives from (1), (2), (3) and (4) which is correct or most appropriate and write your response in the answer box provided.

[50 marks]

1. A program named `sort.c` uses the `scanf` function. Assume that program is compiled using `gcc -o sort sort.c`. Which of the following lists the correct order of executions that will take place due to the above command?

- (1) assembler → preprocessor → linker → cc1
(2) cc1 → linker → preprocessor → assembler
(3) linker → cc1 → preprocessor → assembler
(4) preprocessor → cc1 → assembler → linker

ANSWER:

2. Which of the following statements are true about the `stdio.h` header file?

A- It is an example for a library code.

B- when a C program uses input/output functions like `scanf` and `printf`, this must be included at the top of the program.

C- It is a text file.

- (1) A and B only (2) A and C only (3) B and C only (4) All A, B and C

ANSWER:

♣ Ternary conditional operators can be nested just like if-else statements. In the following C code fragment which is used for questions 3 and 4, assume `a`, `b` and `ans` are defined to be integers.

```
ans = (a == 2 ? (b == 2 ? 3 : 5) : 1);  
printf ("%d\n", ans);
```

3. If `a = 2` and `b = 2` what will be the output of the above C fragment?

- (1) 1 (2) 2 (3) 3 (4) 5

ANSWER:

4. If `a = 1` and `b = 3` what will be the output of the above C fragment?

- (1) 1 (2) 2 (3) 3 (4) 5

ANSWER:

5. What will be the result if the following C program is compiled and executed?

```
#include <stdio.h>

enum Level {POOR, AVERAGE, GOOD=4, EXCELLENT};

int main() {
    // Create enum variables and assign values to them
    enum Level myVar1 = POOR;
    enum Level myVar2 = EXCELLENT;

    printf("%d,%d\n", myVar1, myVar2);
}
```

(1) 0,3 (2) 0,5 (3) 1,4 (4) 1,5

ANSWER: ☐

6. What will be the result if the following C program is compiled and (if possible) executed?

```
#include <stdio.h>

int main() {
    char a;
    a = 'AB';
    printf("a=%c %d\n", a, a);
}
```

(1) compilation error (2) a=A 65 (3) a=B 66 (4) a=AB 6566

ANSWER: ☐

7. What would be the outputs of the following two codes?

```
/* Program 1 */

#include <stdio.h>

int main() {
    int i;
    for (i=0; i<2; ++i)
        printf("%d\t", i);
}
```

```
/* Program 2 */

#include <stdio.h>

int main() {
    int i;
    for (i=0; i<2; i++)
        printf("%d\t", i);
}
```

(1) Program 1: 0 1 Program 2: 0 1 (2) Program 1: 0 1 Program 2: 0 2
(3) Program 1: 0 1 2 Program 2: 0 1 2 (4) Program 1: 0 1 2 Program 2: 0 2 4

ANSWER: ☐

8. Consider the following C statements:

```
int d = 6;
int *p = &d; /* Statement 2*/
int x = *p;
```

Which of the following statements are true?

- A The *Statement 2* assigns 6 to p.
- B The *Statement 2* defines p to be pointer to an integer and puts the address of d as its content.
- C x is a pointer variable.

(1) B only (2) A and C only (3) B and C only (4) All A, B and C

ANSWER:

☐

9. Assume the following C fragment is part of a code named `solve.c`:

```
#ifdef UCSC
/* Statement block 1 */
#else
/* Statement block 2 */
#endif
```

NOTE: Statement blocks named 1 and 2 contain valid C statements.

What will the preprocessor do with respect to the above fragment when the code is compiled using `cc -o solve -DUCSC solve.c`?

- (1) It converts *Statement block 1* to Assembly language.
- (2) It includes *Statement block 1* for compilation.
- (3) It links any library functions used in the code.
- (4) It makes the *solve* executable file.

ANSWER:

☐

10. Which of the following statements are true?

- A- When arrays are declared, their sizes must be indicated.
- B- When a matrix is declared as `int A[2][2];`, the element `A[0][1]` lies after the element `A[0][0]` in memory.
- C- When an array name is passed to a function, the function can carry out processing on the original elements of that array.

(1) A and B only (2) A and C only (3) B and C only (4) All A, B and C

ANSWER:

☐

11. Which of the following lists the given types of variables from the largest to the smallest scope?

- (1) automatic, extern, external
- (2) automatic, external, extern
- (3) extern, external, automatic
- (4) external, extern, automatic

ANSWER:

☐

12. What is the character that `getchar()` returns when there are no more characters in the input buffer?

(1) `\0` (2) `\n` (3) EOF (4) NULL

ANSWER: ☐

13. Which of the following data types consumes the largest storage space?

(1) `char` (2) `int` (3) `float` (4) `double`

ANSWER: ☐

♣ To answer questions 14,15 and 16, consider the following declaration in a program:

```
char title[5]="ABCDE";
```

14. `printf("%d\n",title[2]);` would output

(1) 66 (2) 67 (3) B (4) C

ANSWER: ☐

15. `printf("%c\n",title[2]);` would output

(1) 66 (2) 67 (3) B (4) C

ANSWER: ☐

16. `printf("%s\n",title+1);` would output

(1) 66676869 (2) 6566676869 (3) BCDE (4) ABCDE

ANSWER: ☐

17. Which of the following characters end a string constant?

(1) `\0` (2) `\n` (3) EOF (4) NULL

ANSWER: ☐

18. Which of the following statements are true?

A- '`x`' and "`x`" mean the same.

B- '`x`' is an integer relevant to letter `x` in the machine's character set.

C- "`x`" is stored with a `\n` at its end.

(1) A only (2) B only (3) C only (4) B and C only

ANSWER: ☐

19. If `mp` is a pointer, then which of the following does **not** mean the same thing as the others?

(1) `*mp += 1` (2) `++*mp` (3) `(*mp)++` (4) `*mp++`

ANSWER: ☐

20. If `message` is declared as `char *message;` and the statement `message="Error";` is used, then which will be assigned to `message`?

(1) address of character E (2) address of `message`
(3) the character E (4) the string `Error`

ANSWER: ☐

Part B

1. The following code extract was taken from a pointer based linked list implementation. The included functions are designed to do the work indicated in comments.

Fill the five blanks indicated by ----- to complete the code. The lines where the blanks exist are indicated with comments.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

/* display list of nodes */
void displayList(){
    struct node *current = head;
    printf("\n[");

    while(current != -----) { /* blank 1 */
        printf(" %d ",current->data);
        current = current->next;
    }
    printf("]");
}

/* insert the given integer at the beginning of the list */
void insertAtBegin(int data){

    struct node *new = (struct node*) malloc(sizeof(struct node));

    ----- = data; /* blank 2 */
    new->next = head;
    head = -----; /* blank 3 */
}

/* insert the given integer at the end of the list */
void insertatend(int data){

    struct node *new = (struct node*) malloc(sizeof(struct node));
    new->data = data;
    struct node *current = head;

    while(-----) /* blank 4 */
        current = current->next;

    current->next = new;
    current->next-> ----- = NULL; /* blank 5 */
}
```

[10 marks]

2. The following code extract was taken from a pointer based linked list implementation. The included functions are designed to do the work indicated in comments.

Fill the five blanks indicated by ----- to complete the code. The lines where the blanks exist are indicated with comments.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

/* Delete node at the end of the list */
void deleteatend(){
    struct node *current = head;
    struct node *previous = head;

    if (current != -----) { /* blank 1 */

        if (----- == NULL) { /*just one node in list*/ /* blank 2 */
            free(current);
            head = NULL;
        } else {
            while (1) {
                if (current->next != NULL) {
                    previous = current;
                    current = current->next;
                } else {
                    ----- = NULL; /* blank 3 */
                    free(current);
                    break;
                }
            }
        }
    }

}

/* searches the list for the "key"; returns 1 if existing and 0 if not*/
int searchlist(int key){
    struct node *current = head;
    while(current != NULL) {

        if (-----) { /* blank 4 */
            return 1;
        }
        -----; /* blank 5 */
    }
    return 0;
}
```

[10 marks]

3. A user wants to create a simple file management system using C to manage her list of songs. The system should display a menu with four options as follows:

1. **Add** - The system should get the *singer* and *title* of a song from the user and add that record to the *songs.txt* file.
2. **Display** - The system should display the existing song list in the file by printing the *singer* and *title* of each song.
3. **Select** - The system should print the *title* and *singer* of the *first song* in the file and then remove it from the file and add that record again to the file as its *last record*.
4. **Exit** - The system should stop and give back the command prompt.

An incomplete code of the above system is given below with 10 blanks (each indicated by a -----). The suitable labelled answer choices for the blanks are given later. Write down the label of the correct answer choice for each blank on the relevant ----- . The lines where the blanks exist are indicated by comments.

```
#include <stdio.h>
#include <string.h>

typedef struct
{
    char singer[10];
    char title[50];
} Song;

Song s, selected;

void display(FILE *);

int main() {
    int option;
    FILE *fp;
    FILE *fTemp;

    -----; /* blank 1*/
    printf("My song list\n");
    printf("*****\n");
    printf("\nPress:\n 1-Add\n 2-Display\n 3-Select\n 4-Exit\nYour Option? ");

    while (-----) { /* blank 2 */
        switch(option) {
            case '1': /* add */
                printf("Enter singer,title of song:\n");
                -----; /* blank 3*/
                -----; /* blank 4 */
                break;
```



```

case '2': /* Display */
    printf("\nYour play list:\n");
    display(fp);
    break;

case '3': /* Select */
    rewind(fp);
    if (-----) { /* blank 5 */
        printf("Your song: %s by %s\n",selected.title,selected.singer);

        /* copy remaining structures to temporary file */
        -----; /* blank 6 */
        while(fread(&s,sizeof(s),1,fp))
            -----; /* blank 7 */
        /* copy selected record as the last one*/
        -----; /* blank 8 */
        fclose(fp);
        fclose(fTemp);
        /* Delete original source file */
        remove("songs.txt");
        /* Rename temp file as original file */
        rename("temp.txt", "songs.txt");
        fp=fopen("songs.txt","a+");
    } else
        printf("Nothing in play list!\n");
    break;
} /* switch end */
printf("\n\na?");
}
fclose(fp);
}

void display(FILE *fp) {
    -----; /* blank 9 */
    printf("Singer\tTitle\n");
    while(-----) /* blank 10 */
        printf("%s\t%s\n",s.singer,s.title);
}

```

Label	Answer choice
A	(fread(&selected,sizeof(selected),1,fp)) > 0
B	(option = getchar()) != '4'
C	fp=fopen("songs.txt","a+")
D	fread(&s,sizeof(s),1,fp)
E	fTemp = fopen("temp.txt", "w")

Label	Answer choice
F	fwrite(&s,sizeof(s),1,fp)
G	fwrite(&s,sizeof(s),1,fTemp)
H	fwrite(&selected,sizeof(selected),1,fTemp)
I	rewind(fp)
J	scanf("%s%s",s.singer,s.title)

NOTE: A copy of these choices in larger print are given in Figure 2 on Page 12.

[10 marks]

4. The following incomplete code is designed to use recursion to print the number of digits of an input number n as shown by the example run shown in the figure.

```
$ ./a.out
```

```
The digits of a given number:
*****
```

```
Input number:59317
No. of digits in number 59317 is: 5
```

Fill the **nine** blanks in the code (indicated by -----) to complete it. The lines where the blanks exist are indicated by comments.

```
#include <stdio.h>

int -----(int number); /* blank 1 */

int main() {
    int number, counter;
    printf("\n\nThe digits of a given number:\n");
    printf("*****\n");
    printf("\nInput number:");

    scanf("%d",-----); /* blank 2 */

    counter = -----(number); /*blank 3 */

    printf("No. of digits in number %d is: %d \n\n",-----,-----);
                                                /*blanks 4 and 5 */
    return 0;
}

int noOfDigits(int n1) {
    ----- int ctr=0; /* blank 6 */

    if(n1!=0) {
        ctr++;

        -----(-----); /* blanks 7 and 8 */
    }
    return -----; /* blank 9 */
}
```

[10 marks]

5. (a) Consider the following code:

```
#include <stdio.h>

int main() {
    int c;

    while (c = getchar() != EOF)
        printf("c=%d\n", c);
}
```

When the user compiled and ran the above code and typed 7 followed by the RETURN key and Ctrl-D, the output was as follows:

```
c=1
c=1
```

Why?

.....
.....
.....

[4 marks]

(b) Explain program *indentation* and its benefits.

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

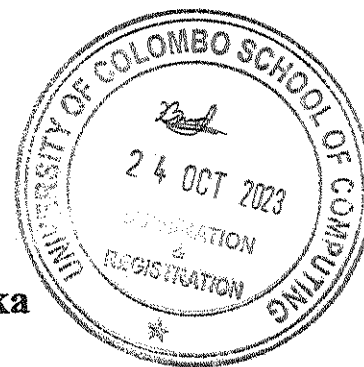
[6 marks]

Operator	Associativity
() [] -> .	left to right
! ~ ++ -- + - * & (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
	left to right
?:	right to left
= += -= *= /= %= &= ^= = <<= >>=	right to left
,	left to right

Figure 1: C operator precedence table

Label	Answer choice
A	(fread(&selected,sizeof(selected),1,fp)) > 0
B	(option = getchar()) != '4'
C	fp=fopen("songs.txt","a+")
D	fread(&s,sizeof(s),1,fp)
E	fTemp = fopen("temp.txt", "w")
Label	Answer choice
F	fwrite(&s,sizeof(s),1,fp)
G	fwrite(&s,sizeof(s),1,fTemp)
H	fwrite(&selected,sizeof(selected),1,fTemp)
I	rewind(fp)
J	scanf("%s%s",s.singer,s.title)

Figure 2: A copy of the answer choices for Question 3 in Part B



University of Colombo, Sri Lanka

University of Colombo School of Computing
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

First Year Examination - Semester I - UCSC AY20 [held in October 2023]

SCS1201 — Data Structures & Algorithms I

(Two (2) Hours)

Answer ALL questions

Number of Pages = 7

Number of Questions = 4

To be completed by the candidate

Index Number:

--	--	--	--	--	--	--	--

261

Important Instructions to candidates:

- I. Students should answer in the medium of English language only using the space provided in this question paper.
- II. Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
- III. Write your index number **CLEARLY** on each and every page of this Question paper.
- IV. This paper consists of four(4) questions on seven(7) pages (including the Cover Page). Note not all questions carry equal marks.
- V. Answer ALL questions.
- VI. Programmable Calculators and any electronic device capable of storing and retrieving text including electronic dictionaries, smartwatches and mobile phones are not allowed.
- VII. Do not tear off any part of this answer book. Under no circumstances may this book, used or unused, be removed from the Examination Hall by a candidate.

To be completed by the examiners

1	
2	
3	
4	
Total	

Question 1(45 marks)

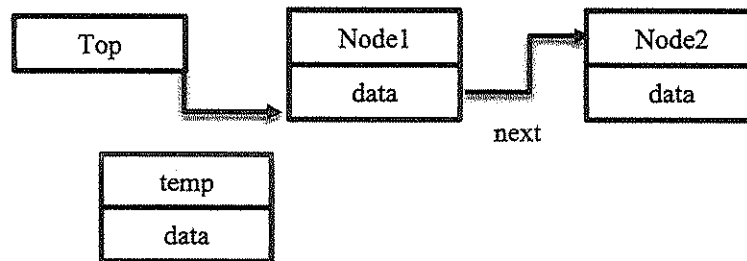
(a) What is the implementation mechanism used for inserting and deleting elements from a stack ?

[2 Marks]

ANSWER IN THIS BOX

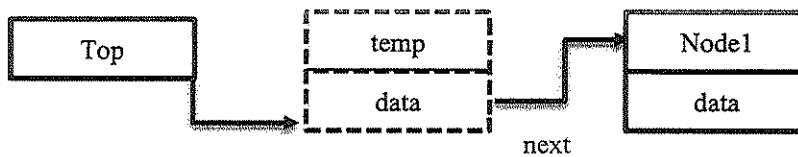
(b) How can you implement the following listed stack operations by using a singly linked list ? Note : you are supposed to mention all requirements to perform below mentioned operations.

1. Use the following illustration to **push temp** node into the linked list. 10 marks]



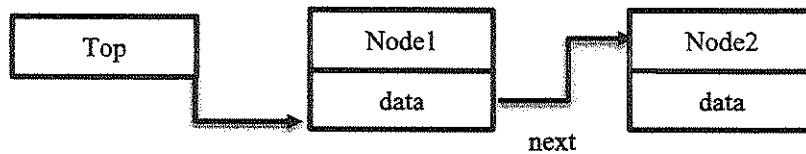
ANSWER IN THIS BOX

2. Use the following illustration to **pop temp** node from the linked list. [10 marks]



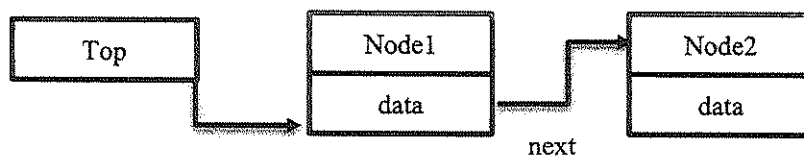
ANSWER IN THIS BOX

3. Use the following illustration to **peek** node from the linked list.[10 marks]



ANSWER IN THIS BOX

4. Use the following illustration to **print** all nodes of the stack using the linked list representation. [10 marks]



ANSWER IN THIS BOX

(c) What are the time and space complexity of the above operations mentioned in section (b)? [3 Marks]

ANSWER IN THIS BOX

Question 2 (25 marks)

- (a) Someone aims to implement a **priority queue** for scheduling tasks in an operating system. Which tree data structure is the most suitable for managing this scenario effectively ?

[5 Marks]

ANSWER IN THIS BOX

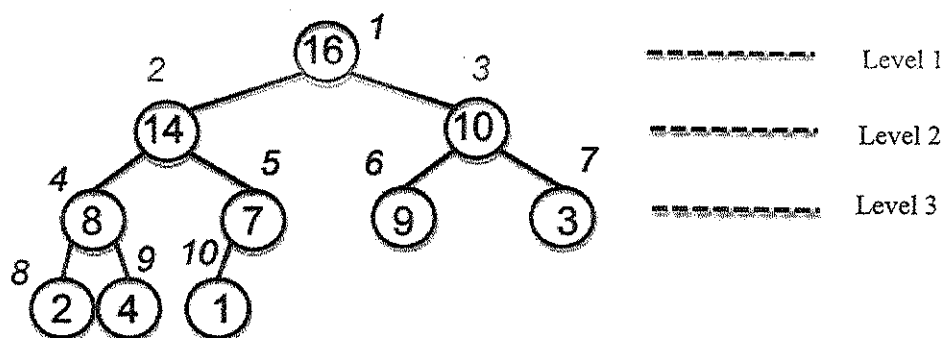
- (b) Consider the following elements as the priority of the tasks.

1	4	2	3	7	8	9	10	14	16
---	---	---	---	---	---	---	----	----	----

We have built a data structure that you can easily track the highest priority tasks (high in value) when exacting it with $O(1)$ complexity.

Note : This tree illustration should return highest priority task as the root of the tree.

Final data structure that consists of highest priority task as the root of the tree is illustrated below.



- (i) If the highest priority task is moved to the processor from the task list and it is not there anymore in the tasks list(node 16 is deleted), show the next available high priority element is the data structure without violating data-structure properties. Draw final data structure that returns high priority task as the root of the tree with no violations until level 3 (including level 3). [15 marks]

ANSWER IN THIS BOX

- (c) What is the sorting time complexity to maintain the above priority tree-based data structure for n elements ?

[5 Marks]

ANSWER IN THIS BOX

Question 3 (15 marks)

- (a) Illustrate how quick sort works for the data array below using a diagram. Clearly show the pivot value and how elements are moved using arrows along with the pivot value.

[5 Marks]

Note : Use the first element as the pivot.

10	23	51	18	4	31	5	13
----	----	----	----	---	----	---	----

ANSWER IN THIS BOX

- (b) How can you make the quick sort algorithm more efficient ?

[5 Marks]

ANSWER IN THIS BOX

- (c) Fill in the missing code for the quick sort algorithm.

[5 Marks]

```
quicksort(L)
{
    if (length(L) < 2) return L
    else
    {
        pick some x in L    // x is the pivot element

    }
}
```

ANSWER IN THIS BOX

Question 4 (15 marks)

Consider the following code segment.

int sum = 0;	C1
for(int i = 0; i < n ; i++){	C2
for(int j = 0; j < n ; j++){	C3
for(int k = 0; k < n ; k++){	C4
sum +=arr[i][j][k];	C5
k=k+2;	C6
}	
}	
}	

- (a) Calculate the code segments running time considering its cost.

Note : Total running time = $\sum_{k=0}^n \text{costK} * \text{running time of line K}$

[10 Marks]

ANSWER IN THIS BOX

- (b) What is the time complexity of the above code segment in terms of Big-O ?

[5 Marks]

ANSWER IN THIS BOX
